

Введение в Python

Гейне М.А.

14.09.2026

Что такое Python?

Python - это высокоуровневый язык программирования общего назначения, известный своей простотой, читабельностью и универсальностью. Он широко используется в различных областях, включая веб-разработку, науку о данных, машинное обучение, автоматизацию и научные вычисления.

История Python

Pre-Python

- Конец 1980-х. Голландский программист **Гвидо ван Россум** работает в институте Centrum Wiskunde & Informatica, CWI
- Разрабатывает распределённую ОС Amoeba
- Под рукой только sh и C
- В этом же институте разрабатывается учебный язык ABC
- Идея - сделать язык с читаемостью и простотой ABC, но с применимостью C

Python 0.9.0

- Февраль 1991: Python 0.9.0 опубликован в группе `alt.sources`
- Практический и расширяемый язык сценариев, унаследовавший читаемость ABC
- Классы с наследованием, исключения, функции, `list`, `dict`, `str` и прочие базовые типы
- Система модулей
- В 1994 появилась группа `comp.lang.python`: сообщество стало частью развития языка

Fun Fact

Python - это не в честь змеи, а в честь шоу "Летающий цирк Монти Пайтона" (Monty Python's Flying Circus)

Январь 1994, Python 1.0

- `lambda`, `map`, `filter`, `reduce` : функциональные средства, вдохновлённые Lisp
- 1.4: именованные аргументы, комплексные числа, искажение имён атрибутов
- 1.5: пакеты модулей и ключевое слово `assert`
- 1.6: поддержка Unicode и модуль `distutils`
- 1999: инициатива Computer Programming for Everybody (CP4E)

Зачем понадобился Python 2?

- Python 1.x вырос из небольшого проекта в язык с международным сообществом
- Понадобилось развить Unicode, управление циклическими ссылками и выразительную работу с коллекциями
- Разработка становилась более открытой: изменения обсуждались и документировались публично
- Python 2 был эволюционным релизом, а не перепроектированием языка

Октябрь 2000, Python 2.0

- List comprehensions и сборка циклического мусора — сильное влияние функциональных языков
- Unicode и более открытая модель разработки
- Декабрь 2001, Python 2.2, унификация типов и классов
- Сентябрь 2006, Python 2.5: контекстные менеджеры и оператор `with`

Подготовка к Python 3

- Python 2.6 переносил часть возможностей 3.0 назад и предупреждал о несовместимостях
- `__future__`, ключ `-3` и инструмент `2to3` помогали готовить код к миграции
- Python 2.7 стал последней веткой 2.x; срок поддержки продлили до 2020 года
- Последний релиз, Python 2.7.18, состоялся 20 апреля 2020 года

3 декабря 2008, Python 3.0

- Чётко разделены текст (`str`) и байты (`bytes`)
- Устранены дублирующие и исторически неудачные интерфейсы
- `range()` стал компактным объектом диапазона, `/` — обычным делением, `print()` — функцией
- Цена упрощения языка — намеренная несовместимость с Python 2

Проблемы совместимости

- Смена версий не прошла безболезненно. Были средства автоматизации (например `2to3`), но они справлялись с задачей лишь частично
- Изначальная схема совместимости: держите всю кодовую базу на Python 2, а для Python 3 делайте дополнительную версию с помощью `2to3`, и изменения туда лучше не вносить. Получалось две кодовые базы.
- К началу 2010-х многие проекты перешли к одной кодовой базе для Python 2.7 и 3.x; различия скрывали модули совместимости, например `six`
- К примеру, `six.string_types` в 2 версии был `(str, unicode)`, а в 3 - `str`. `isinstance(my_var, six.string_types)` за счёт этого работал одинаково в обеих версиях языка

Основные фишки Python 3

- `print` стал функцией, а не оператором: его проще передавать и заменять
- `raw_input()` стал `input()`; прежний вычисляющий выражение `input()` удалён
- `reduce()` перенесён из встроенного пространства имён в `functools`
- Появился синтаксис аннотаций функций; типизация осталась динамической

Основные фишки Python 3

- `str` стал типом Unicode-текста; для последовательностей байтов появился `bytes`
- Строки и байты больше нельзя неявно смешивать
- `/` всегда выполняет обычное деление: `9 / 2 == 4.5` ; для целой части есть `//`
- Идентификаторы и исходный код полноценно поддерживают Unicode
- Удалён ряд особенностей, сохранявшихся только ради совместимости

Другие крупные релизы после 3.0

- 3.6: f-строки, аннотации переменных, асинхронные генераторы и comprehensions
- 3.7: `dataclasses`, `contextvars`, `breakpoint()`, гарантированный порядок `dict`
- 3.9: объединение словарей `|`, типы вида `list[str]`, `zoneinfo`, PEG-парсер
- 3.10: structural pattern matching, объединения типов `X | Y`, точные номера строк

Другие крупные релизы после 3.0

- 3.11: Faster CPython, `ExceptionGroup` и `except*`, более точные traceback
- 3.12: per-interpreter GIL через C API, параметры типов, полная грамматика f-строк
- 3.13: новый REPL, экспериментальные free-threaded сборка без GIL и JIT
- 3.14: free-threaded сборка официально поддерживается, но остаётся опциональной; JIT всё ещё экспериментален

Fun Fact

Долгое время Гвидо ван Россум носил титул **BDFL** — «Великодушный пожизненный диктатор» (Benevolent Dictator For Life)

В 2018 году он отказался от этой роли. Теперь окончательные решения принимает **Steering Council** из 5 человек

Кто стоит за Python?

- **Core team** разрабатывает и рецензирует язык и CPython
- **PEP** фиксируют мотивацию, спецификацию и обсуждение крупных изменений
- **Steering Council** принимает окончательные технические решения
- **Python Software Foundation** поддерживает инфраструктуру, сообщество и юридическую основу проекта

Философия Python

- Readability counts
- Explicit is better than implicit
- Simple is better than complex
- Errors should never pass silently
- Practicality beats purity
- There should be one — and preferably only one — obvious way to do it

Ключевые особенности Python

- **Удобство чтения:** Синтаксис Python разработан так, чтобы его было легко читать и понимать, что делает его отличным выбором для новичков.
- **Универсальность:** Python можно использовать для решения широкого круга задач, от простых скриптов до сложных приложений.
- **Большое сообщество:** Python имеет большое и активное сообщество, предоставляющее обширную документацию, библиотеки и поддержку

Ключевые особенности Python

- **Кроссплатформенная совместимость:** Python работает на различных операционных системах, включая Windows, macOS и Linux.
- **Динамическая типизация:** тип принадлежит объекту, а одно имя в разное время может быть связано с объектами разных типов; проверки происходят во время выполнения.
- **Разные стратегии исполнения:** Python не предписывает одну реализацию. CPython компилирует исходный код в байткод и исполняет его на виртуальной машине; другие реализации могут применять JIT.

Реализации Python

1. **CPython**. Основная реализация на C: исходник → байткод → виртуальная машина
2. **PyPy**. Альтернативная реализация с tracing JIT для «горячего» Python-кода
3. **MicroPython**. Компактная реализация для микроконтроллеров
4. **Jython / IronPython**. Интеграция Python с JVM / .NET
5. **GraalPy**. Реализация на GraalVM с JIT и интеграцией с Java

Применение Python

- Data Science and Machine Learning
 - NumPy, pandas и Matplotlib — основные средства для обработки, анализа и визуализации данных
 - scikit-learn, TensorFlow, Keras, PyTorch — инструменты для построения и обучения моделей
- Scientific Computing
 - SciPy и SymPy - численные симуляции и математические исчисления
 - Matplotlib - визуализация научных данных

Применение Python

- Веб-разработка
 - Django, Flask, FastAPI — фреймворки для разработки веб-приложений и API
 - BeautifulSoup и Scrapy — инструменты извлечения данных с веб-страниц
- Образование
 - Python создавался как практический язык, но благодаря читаемости стал одним из основных языков обучения программированию

Применение Python

- Автоматизация
 - Простой скриптовый язык позволяет достаточно легко и быстро автоматизировать ручные рутинные задачи
- Системы управления
 - Home Assistant - система управления умным домом с открытым исходным кодом, работающая в основном на Python
 - Интеграции с умными устройствами пишутся на Python и доступны любому для редактирования

Пример анализа данных

0. Подготовка датасета

ФИО	Срок постановки на учет в Ж/К	Дата постановки на учет в центре	Способ родоразрешения	Х/П с какого срока	ВН в 36 недель	Дата родов	Срок беременности	Х/п ребенку	Примечания	Год
[REDACTED]	Не состояла	24.04.2016	Самостоят роды	Однократно со срока 18 нед, назначена в МРЦКБ	неизвестная	01.01.2019	39	3 преп, отказ от ребенка	Не наблюдалась, Бродяжничала.Активный ПИН	2019
[REDACTED]	27.07.2018-8 недель	03.08.2007	к/с плановое	Арвт до беременности	Отрицательная	17.01.2019	38	1 препарат		2019
[REDACTED]	01.08.2018-12 нед	21.07.2010	к/с плановое	34 нед	Положительная	07.02.2019	39-40	3 преп	ВИЧ-диссидентка	2019
[REDACTED]	08.02.2019-33 нед	28.05.2015	к/с экстренное	АРВТ до беременности	Положительная	11.02.2019	34	3 препарат, отказ от ребенка	Не наблюдалась, Бродяжничала.Активный ПИН ,коммерческая секс работница	2019
[REDACTED]	17.08.2018 12 нед	06.03.2017	к/с плановое	Арвт до беременности	Отрицательная	12.02.2019	38-39	3 препарат	Глубокая иммуносупрессия	2019
[REDACTED]	21.08.2018-12 нед	14.05.2013	к/с плановое	С 14 нед	Отрицательная	24.02.2019	38-39	1 препарат		2019
[REDACTED]	10.10.2018-16 нед	20.10.2011	к/с экстренное	С 18 нед	Отрицательная	25.02.2019	38 нед	3 препарат		2019
Горькова	20.08.2019-11 нед	02.06.2010	к/с плановое	С 37 нед	Отрицательная	05.03.2019	38-39	3 препарат	Не наблюдалась в Центре ,не направлена ж/к	2019

1. Загрузка датасета

```
df = pd.read_excel('Dataset2.xlsx')

date_columns = ['OBGYN', 'AIDS', 'BirthDate']

for col in date_columns:
    df[col] = pd.to_datetime(df[col], format='%d.%m.%Y', errors='coerce')
```

	Name	Week	AIDS	BirthType	Therapy	Renewed	Status	Length	Dosage	Comments	Year
0	Дав	39	-709	сам	-2	0.0	1.0	39	3.0	Не наблюдалась, Бродяжничала.Активный ПИН, отказ от ребенка	2019
1	Мак	8	-3919	план	0	0.0	0.0	38	1.0	0.0	2019
2	Тим	12	-2843	план	34	0.0	1.0	40	3.0	ВИЧ-диссидентка	2019
3	Фил	33	-1117	план	0	0.0	1.0	34	3.0	Не наблюдалась, Бродяжничала.Активный ПИН ,коммерческая секс работница, отказ от ребенка	2019
4	Сел	12	-435	план	0	0.0	0.0	39	3.0	Глубокая иммуносупрессия	2019
5	Ива	12	-1839	план	14	0.0	0.0	39	1.0	0.0	2019
6	Гал	16	-2419	экстр	18	0.0	0.0	38	3.0	0.0	2019
7	Гор	11	-2925	план	37	0.0	0.0	39	3.0	Не наблюдалась в Центре ,не направлена ж/к	2019
8	Ерм	8	-1368	план	14	0.0	0.0	39	1.0	0.0	2019
9	Кар	15	89	план	18	0.0	0.0	38	3.0	Тяжелый гепатоз беременных	2019

2. Трансформация данных

```
df['LengthDelta'] = pd.to_timedelta(df['Length'], unit='W')
df['PregnancyStart'] = df['BirthDate'] - df['LengthDelta']

df['OBGYN'] = df['OBGYN'].fillna(df['BirthDate'])
df['AIDS'] = df['AIDS'].fillna(df['BirthDate'])

missing_week = df[df['Week'] == '-']
missing_week['Week'] = missing_week['Length'] - (pd.to_timedelta((missing_week['BirthDate']
    * missing_week['OBGYN'])).dt.days // 7)
df.update(missing_week)
df['Week'] = df['Week'].astype(int)

# Initialize LabelEncoder
label_encoder = LabelEncoder()

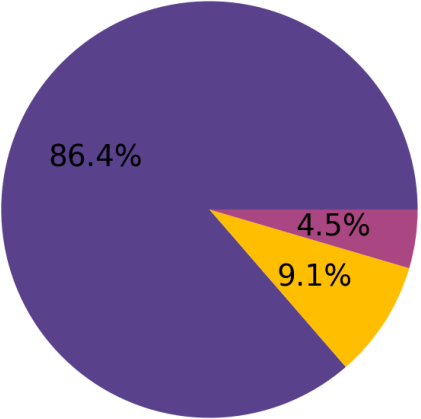
# Fit and transform the 'BirthType' column
df['BirthType'] = label_encoder.fit_transform(df['BirthType'])
```

2. Трансформация данных

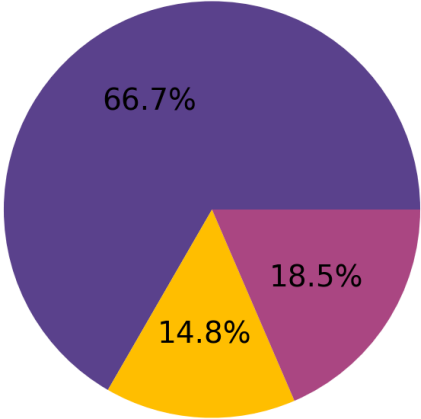
	Week	AIDS	BirthType	Therapy	Renewed	Status	Length	Dosage	PoorCommitment	SocialRisk	Migration	FSIN	Illness	Complications	Death
0	39	-709	1	-2	0.0	1.0	39	3.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0
1	8	-3919	0	0	0.0	0.0	38	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	12	-2843	0	34	0.0	1.0	40	3.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0
3	33	-1117	0	0	0.0	1.0	34	3.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0
4	12	-435	0	0	0.0	0.0	39	3.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
5	12	-1839	0	14	0.0	0.0	39	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
6	16	-2419	2	18	0.0	0.0	38	3.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
7	11	-2925	0	37	0.0	0.0	39	3.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0
8	8	-1368	0	14	0.0	0.0	39	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
9	15	89	0	18	0.0	0.0	38	3.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0

3. Визуализация данных

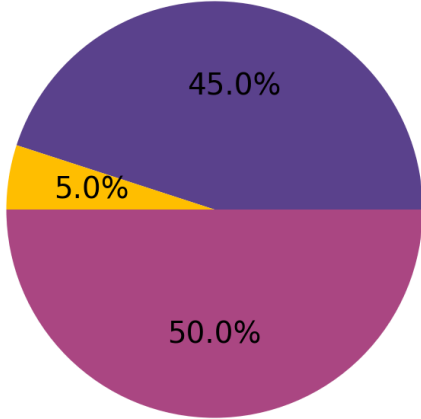
2019 год



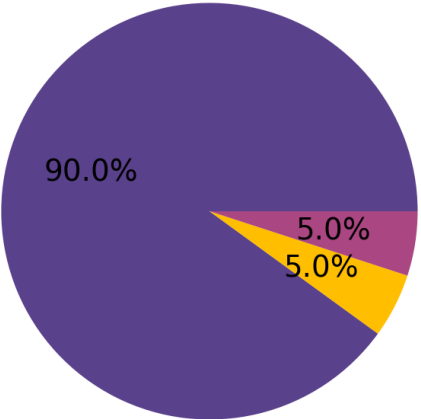
2020 год



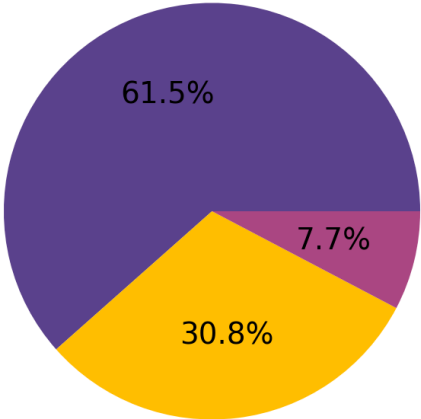
2021 год



2022 год



2023 год



3. Визуализация данных

```
# Get yearly data
yearly_percentages = df.groupby('Year')['BirthType'].value_counts(normalize=True).sort_index() * 100

# Create a single figure with 5 subplots
fig, axs = plt.subplots(2, 3, figsize=(15, 10))

# Plot pie charts for each year
for i, (year, percentages) in enumerate(yearly_percentages.groupby(level=0)):
    # birth_types = [label[1] for label in percentages.index]
    wedges, texts, autotexts = axs[i//3][i%3].pie(percentages, autopct='%1.1f%%', colors=generate_colors(len(percentages)))
    axs[i//3][i%3].set_title(f'{year} год')
axs[-1, -1].legend(wedges, labels, loc='center', fontsize=14)
axs[-1, -1].axis('off')
plt.savefig('fig2.png', transparent=True)
plt.show()
```

4. Разбиение данных на выборки

```
X = df.drop(columns=['Dosage'])
y = df['Dosage']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=df['Dosage'])

print("X train (features):")
print(X_train.head())
print("\nX test (features):")
print(X_test.head())
print("\ny train (target variable):")
print(y_train.head())
print("\ny test (target variable):")
print(y_test.head())
```

4. Разбиение данных на выборки

X train (features):

	Week	AIDS	BirthType	Therapy	Renewed	Status	Length	PoorCommitment	\
69	7	-5036	0	8	1.0	0.0	39	0.0	
101	38	291	0	39	0.0	1.0	39	0.0	
23	9	-536	0	0	0.0	0.0	38	0.0	
51	11	-2024	2	0	0.0	0.0	35	0.0	
19	30	-328	0	0	1.0	1.0	30	0.0	
	SocialRisk	Migration	FSIN	Illness	Complications	Death			
69	0.0	0.0	0.0	0.0	0.0	0.0	0.0		
101	0.0	1.0	0.0	0.0	0.0	0.0	0.0		
23	0.0	0.0	0.0	0.0	0.0	0.0	0.0		
51	0.0	0.0	0.0	0.0	0.0	0.0	0.0		
19	0.0	0.0	0.0	1.0	0.0	0.0	0.0		

X test (features):

	Week	AIDS	BirthType	Therapy	Renewed	Status	Length	PoorCommitment	\
85	12	-4744	0	0	0.0	0.0	41	0.0	
86	9	-3735	2	0	0.0	0.0	28	0.0	
67	17	124	2	17	0.0	0.0	33	0.0	
16	10	-3953	0	0	0.0	0.0	39	0.0	
37	9	-3170	0	0	0.0	0.0	39	0.0	
	SocialRisk	Migration	FSIN	Illness	Complications	Death			
85	0.0	0.0	0.0	0.0	0.0	0.0	0.0		
86	0.0	0.0	0.0	0.0	0.0	0.0	0.0		
67	0.0	0.0	1.0	0.0	0.0	0.0	0.0		
16	0.0	0.0	0.0	0.0	0.0	0.0	0.0		
37	0.0	0.0	0.0	0.0	0.0	0.0	0.0		

```
y_train (target variable):  
69      3.0  
101     3.0  
23      1.0  
51      3.0  
19      3.0  
Name: Dosage, dtype: float64  
  
y_test (target variable):  
85      1.0  
86      1.0  
67      3.0  
16      1.0  
37      1.0  
Name: Dosage, dtype: float64
```

5. Обучение

```
# Create a StandardScaler object
scaler = StandardScaler()

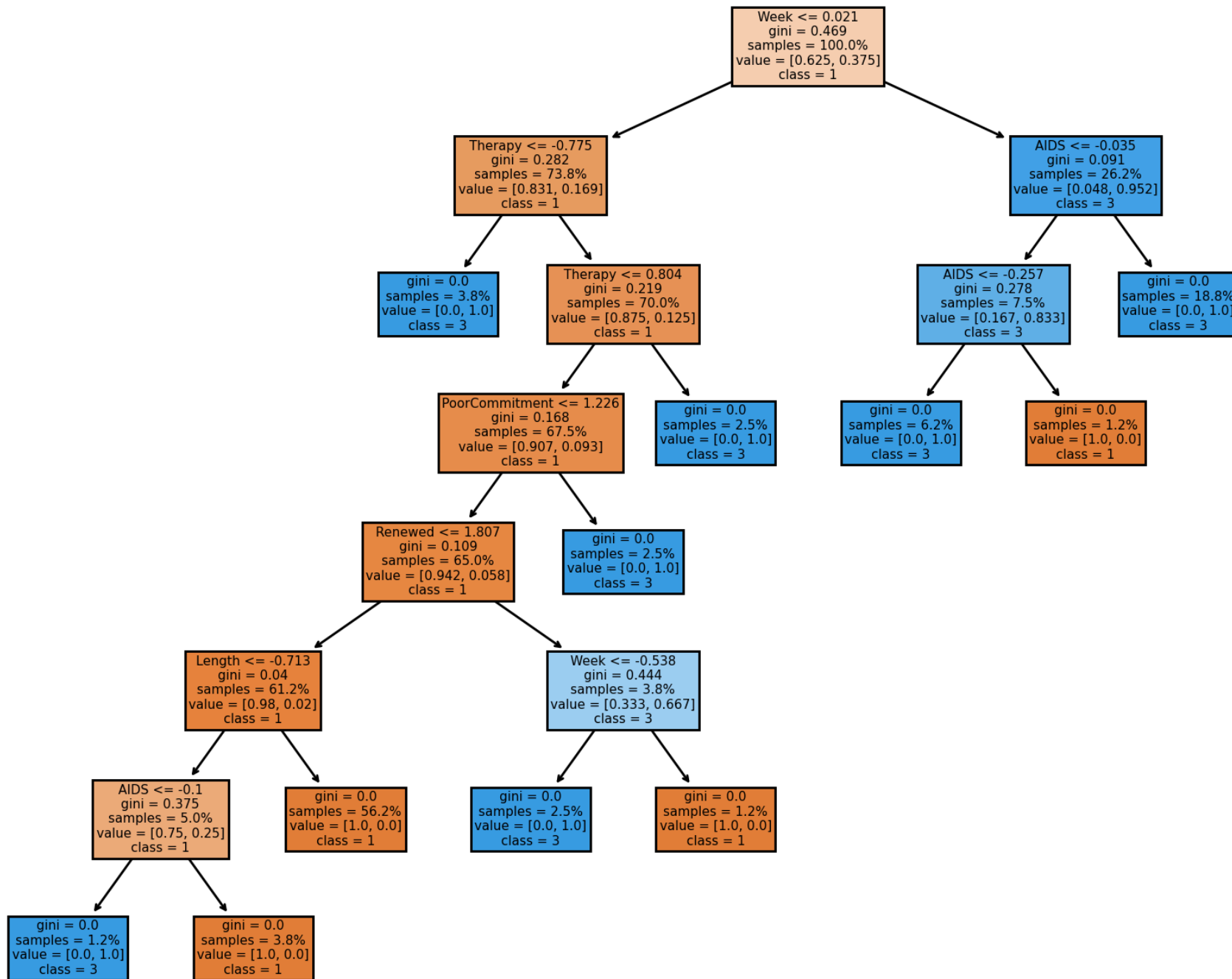
# Fit the scaler on the training data
scaler.fit(X_train)

# Transform the training and testing data
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Instantiate the DecisionTreeClassifier with optional hyperparameters
tree_classifier_scaled = DecisionTreeClassifier(max_depth=50)

# Fit the model to the training data
tree_classifier_scaled.fit(X_train_scaled, y_train)

# Make predictions on the testing data
predictions_scaled = tree_classifier_scaled.predict(X_test_scaled)
```





**Спасибо за
внимание!**